

Running Spectronaut on a Public Dataset in a Distributed Manner

Compatible with Spectronaut v. ≥ 20.3

Grzegorz Skoraczynski

Updated 03.12.2025

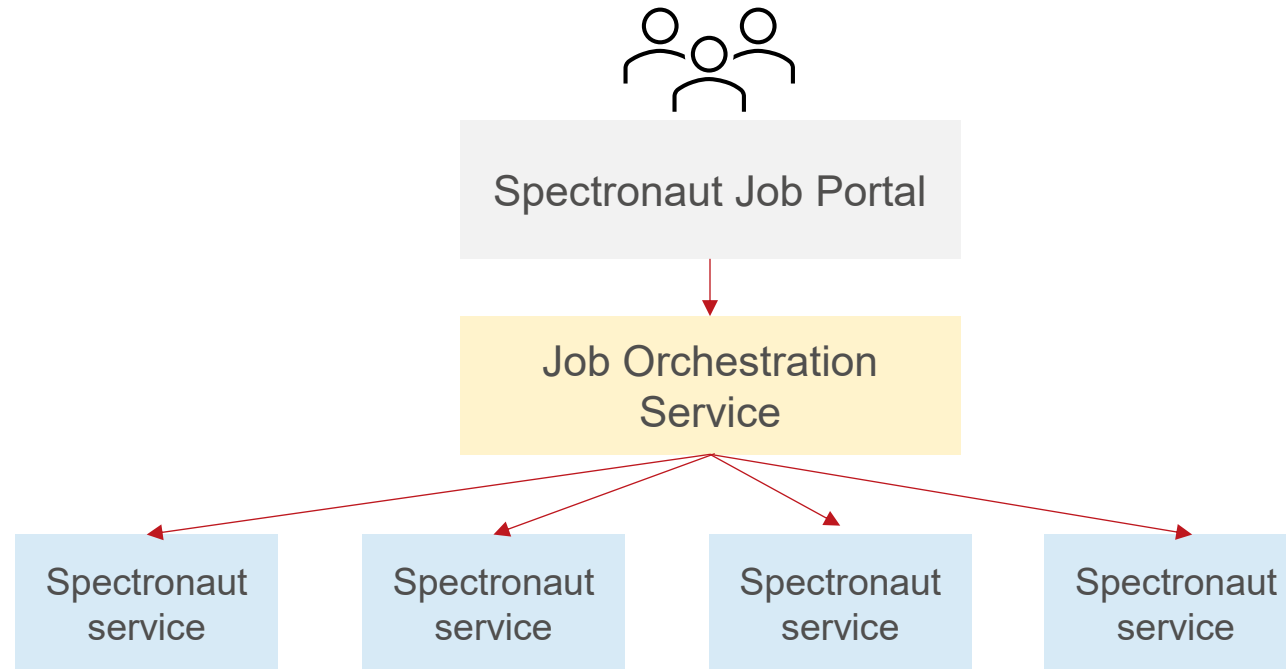
g.skoraczynski@biognosys.com

www.biognosys.com

- > Installing Spectronaut on Linux
- > Preparation of the dataset
- > Spectronaut analysis with a selected dataset
- > Multistep parallelization of Spectronaut with a selected dataset
- > Additional notes

Motivation

- Goal is to motivate how to use Spectronaut in a distributed computing environment for significantly faster computation. Focus of the slides is on directDIA, but same can be achieved for library-based workflow with minor changes.
- This works best in conjunction with a distributed compute architecture as illustrated below:



Installing Spectronaut on Linux

Pre-requisites

- ❖ We support Ubuntu $\geq$ 22.04 LTS. However, Spectronaut has been shown to work with other distributions.
- ❖ For running Spectronaut, the user should have *write* and *read* access to the home directory (/home/<username>/)
- ❖ Nodes or computers where Spectronaut will be run should be able to connect to the Biognosys license server
- ❖ This slide deck assumes that the user has a floating license type for Spectronaut (i.e., you do not need to deactivate Spectronaut when dynamically killing a Spectronaut service). Please contact Biognosys for more information.

Installing Spectronaut on Ubuntu 22.04

Install dependencies

❖ Install .NET 8. For Ubuntu 22.04, type:

```
$ sudo apt-get update  
$ sudo apt-get install -y dotnet-runtime-8.0
```

❖ For older versions of Ubuntu, please install .NET 8 library manually following the instructions: <https://learn.microsoft.com/en-us/dotnet/core/install/linux-ubuntu>

Installing Spectronaut on Ubuntu 22.04

Installing Spectronaut

❖ Download the latest Spectronaut installer

❖ And install:

```
$ sudo dpkg -i Spectronaut_<version>.deb
```

❖ After successful installation, Spectronaut should be available as a `spectronaut` command, e.g.

```
$ spectronaut -h
```

❖ Alternatively, a `tgz` installer can be used. It does not require `sudo` permissions. After unpacking an installer, folder `binaries` and file `read_me.txt` will be created. To run Spectronaut, type:

```
$ dotnet binaries/Spectronaut/bin/SpectronautCMD.dll -h
```

Activate Spectronaut

❖ For every machine, Spectronaut needs to be activated once before using:

```
$ spectronaut activate <license_key>  
Spectronaut successfully activated!
```

To activate, Spectronaut needs an active Internet connection, and tries to connect to the address <https://licensing.biognosys.com/>.

Preparation of a dataset

- ❖ Originating from a paper:

Farabegoli F, Santaclara FJ, Costas D, Alonso M, Abril AG, Espiñeira M, Ortea I, Costas C. Exploring the Anti-Inflammatory Effect of Inulin by Integrating Transcriptomic and Proteomic Analyses in a Murine Macrophage Cell Model. *Nutrients*. 2023; 15(4):859.

- ❖ Analysis of inulin prebiotic activity

- ❖ A murine macrophage cell model (RAW 264.7) of inflammation used

- ❖ RAW 264.7 macrophages cultured under three different conditions:

- ❖ LPS (Escherichia coli lipopolysaccharides),
- ❖ LPS followed by inulin,
- ❖ LPS followed by hydrocortisone,
- ❖ control.

The dataset

Organism	Mouse
Conditions	• L – LPS (Escherichia coli lipopolysaccharides), • IL – LPS followed by inulin, • HL – LPS followed by hydrocortisone, • control
Samples	27 samples in total
Instrument	• timsTOF Pro with diaPASEF method • nanoElute LC, 45 min gradient
PRIDE repository identifier	PXD032759

Obtain data files

❖ Create required directories

```
$ mkdir PXD032759_analysis  
$ cd PXD032759_analysis  
$ mkdir raw other_files results fasta intermediate
```

❖ Download the data from the PRIDE FTP server into a raw directory and unzip it (Warning! It would require about 90 GB of free disk space):

```
$ wget -nd -m ftp://ftp.pride.ebi.ac.uk/pride/data/archive/2023/03/PXD032759/ -P raw  
$ cd raw  
$ for file in *.zip; do unzip $file && rm $file; done  
$ cd ..
```

❖ The command above will download the FASTA files. Replace them for clarity of folder structure

```
$ mv raw/*.fasta fasta/
```

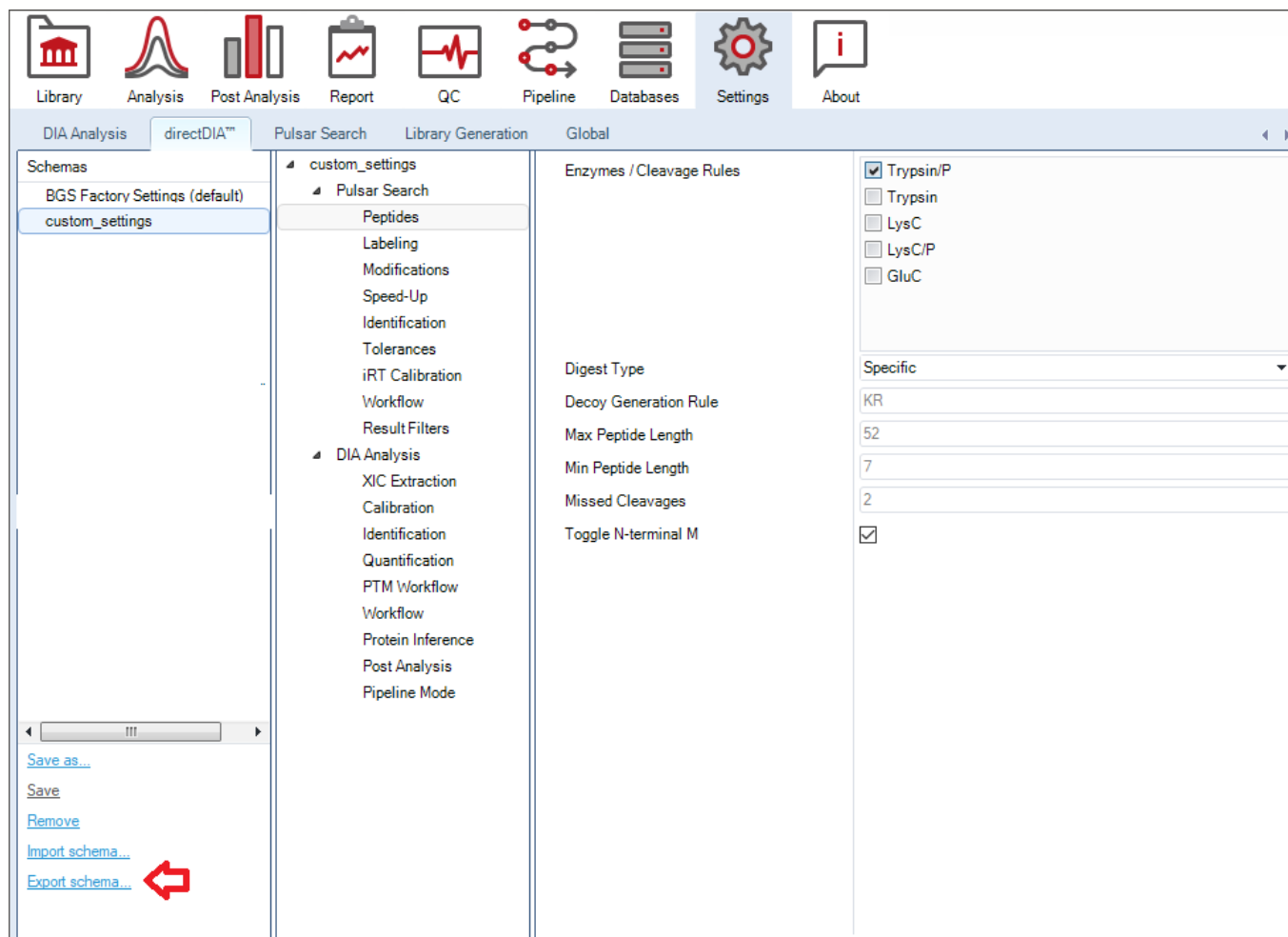

Spectronaut analysis with a selected dataset

Obtain additional files

❖ Before running Spectronaut, download the files required for the correct experiment

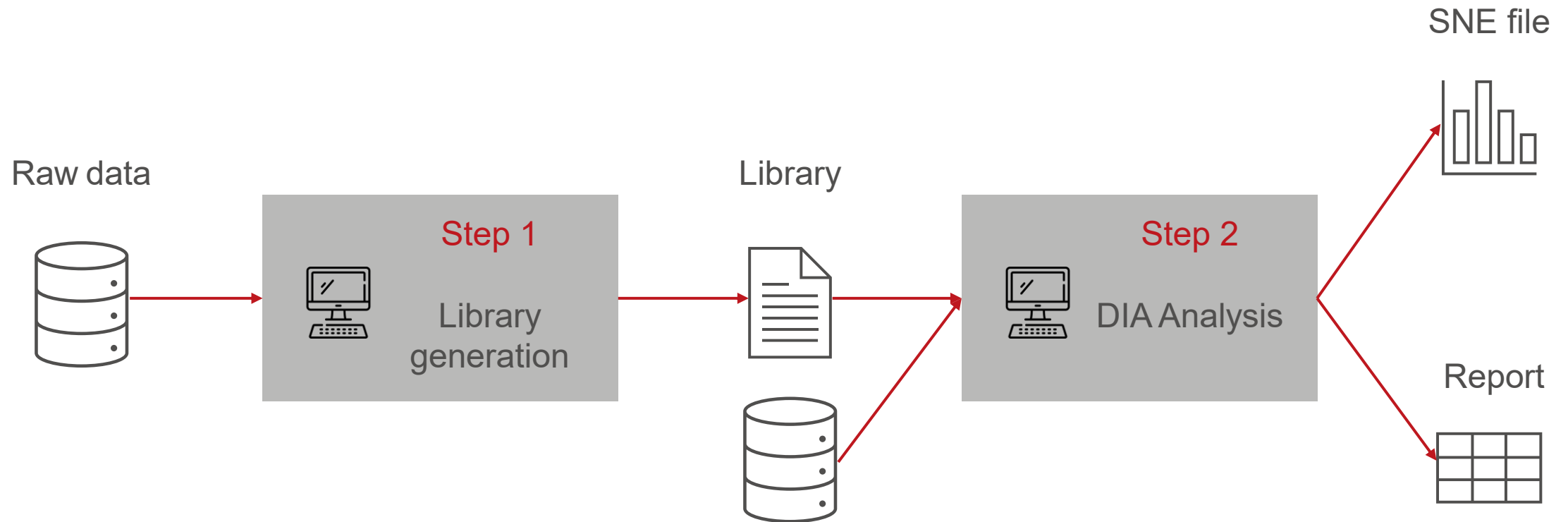
```
$ wget https://files.biognosys.ch/FileSharing/FUKTgiAxv1WzIYrM/PXD032759_condition_setup_full.tsv -P  
other_files
```


Exporting directDIA settings



- ❖ We recommend using custom settings for all analysis steps bundled within single directDIA settings schema
- ❖ To export directDIA settings as a .prop file, open the Settings perspective and select the directDIA tab
- ❖ To export settings schema as a prop file, select the [Export schema...](#) link

directDIA workflow basic concept



Basic commands for directDIA (without parallelization)

Step 1

Create a library using the `lg` command:

```
$ spectronaut lg -se Pulsar -d raw -fasta fasta/20210709-SP_mouse_isoforms_UP000000589.fasta  
-fasta fasta/crap.2009.05.01.fasta -o intermediate  
-a intermediate/search_archive.psar -k intermediate/output_library.kit  
-rs settings/custom_settings.prop -n "PXD032759"
```

The command above:

- Uses the Pulsar search engine to generate a search archive named `search_archive.psar` and places it in the `intermediate` folder (specified with the `-a` option).
- Generates a search library `output_library.kit` and places it in the `intermediate` folder (specified with the `-k` option).
- Analyses all raw files located in the `raw` folder. Input raw files can be specified separately using the `-r` option. Please run `$ spectronaut lg -h` for more details.
- Uses FASTA files specified with the `-fasta` option.
- Uses analysis settings as specified in `settings/custom_settings.prop` file.

Basic commands for directDIA (without parallelization)

Step 2

Do a Spectronaut search of raw files against the library

```
$ spectronaut diaanalysis -o results -a intermediate/output_library.kit -d raw  
-n "PXD032759" -s settings/custom_settings.prop -con other_files/PXD032759_condition_setup_full.tsv
```

The command `diaanalysis` above:

- Searches raw files against the library from **Step 1**. Instead of a library, a search archive can be provided using `-sa intermediate/search_archive.psar` option.
- The experiment is named PXD032759.
- Runs search with default settings schema and default report schema.
- Analyses all raw files located in the raw folder. Input raw files can be specified separately using the `-r` option. Please run `$ spectronaut diaanalysis -h` for more details.
- Uses analysis settings and library generation settings as specified in `settings/custom_settings.prop` file
- Uses `other_files/PXD032759_condition_setup_full.tsv` condition file for differential abundance analysis.
- Results are placed in a time-stamped subfolder of a results folder (`results/<timestamp>_<experiment_name>`).

Basic commands for directDIA (without parallelization)

Summary

For directDIA without parallelization, **Step 1** and **Step 2** can be simplified to a single pipeline

```
$ spectronaut direct -o results -d raw -fasta fasta/20210709-SP_mouse_isoforms_UP000000589.fasta  
-fasta fasta/crap.2009.05.01.fasta -n "PXD032759" -s settings/custom_settings.prop  
-con other_files/PXD032759_condition_setup_full.tsv
```

The command `direct` above:

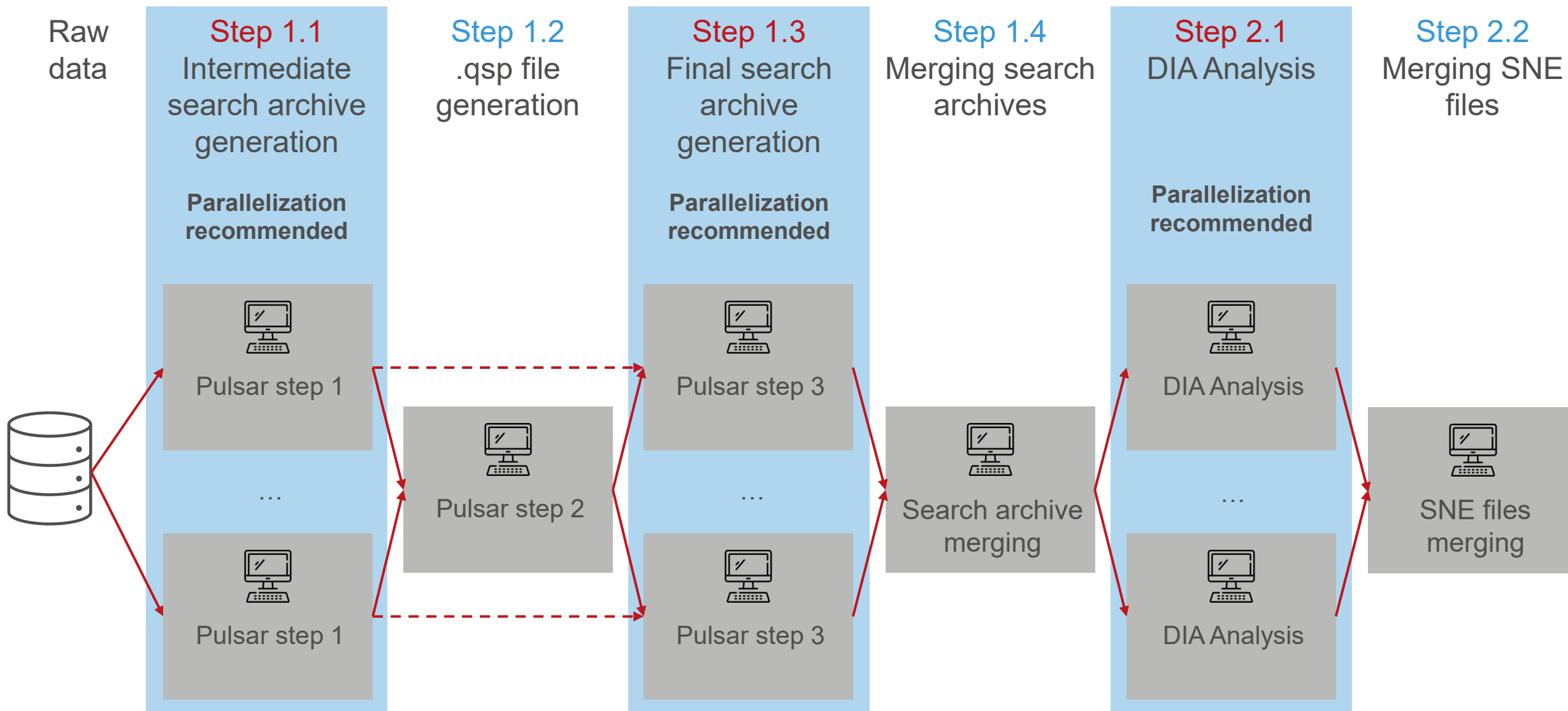
- Does the directDIA search pipeline on raw files in the raw folder. Input raw files can be specified separately using the `-r` option. Please run `$ spectronaut diaanalysis -h` for more details.
- The experiment is named PXD032759, and FASTA files are used as specified,
- Runs search with default settings schema and default report schema,
- Uses analysis settings as specified in `settings/custom_settings.prop` file
- Results are placed in a time-stamped subfolder of a results folder (`results/<timestamp>_<experiment_name>`).
- Uses `other_files/PXD032759_condition_setup_full.tsv` condition file for differential abundance analysis.

- ❖ Results are saved in a subfolder of the `results` folder named `<timestamp>_<name>`
- ❖ It contains among others:

File	Description
PXD032759_Report_BGS Factory Report (Normal).tsv	Spectronaut search results report
PXD032759_AnalysisOverview.txt	Summary of the search results
RunIdentifications.tsv	Number of identifications separately for every run
PXD032759_AnalysisLog.txt	Log from analysis
PXD032759_ExperimentSetupOverview_BGS Factory Settings.txt	Summary of used settings
<timestamp>_PXD032759.sne	Spectronaut experiment file which can be loaded into Spectronaut Viewer for the results visualization
Peptide CVs_CVsBelowX.tsv, Precursor CVs_CVsBelowX.tsv, Protein CVs_CVsBelowX.tsv, Protein Group CVs_CVsBelowX.tsv	The most important post-analysis results. Can be adjusted using settings.

Multistep parallelization of Spectronaut with a selected dataset

Workflow for parallelization of directDIA



Parallelization workflow details

Intermediate search archive generation

To decrease computation time, the Spectronaut search can be split into steps, parallelized within batches. Results are independent of the batch size.

Step 1.1

Create an intermediate search archive:

```
$ spectronaut lg -se Pulsar -r raw/<raw_path_i> ...  
-fasta fasta/20210709-SP_mouse_isoforms_UP000000589.fasta -fasta fasta/crap.2009.05.01.fasta  
-a intermediate/search_archive_step1_file<i>.psar -o intermediate  
-rs settings/custom_settings.prop --pulsarStage pulsarStep1
```

The command above:

- Does the Pulsar step 1 part of the Pulsar search (specified with option `--pulsarStage pulsarStep1`) to generate an intermediate search archive named `search_archive_step1_file<i>.psar`, where `<i>` is the batch number
- Takes one or more raw files for batch `<i>`: `raw/<raw_path_i>` enumerated as an input.
- Raw files can also be specified with the `-d raw` option by putting only batch-specific files in a raw folder.
- Other options as previously described.

Parallelization workflow details

A .qsp file generation

Step 1.2

All intermediate search archives are collected to generate a .qsp file:

```
$ spectronaut lg -se Pulsar -sa intermediate/search_archive_step1_file1.psar  
-sa intermediate/search_archive_step1_file2.psar ...  
-o intermediate/qsp --noOutputSubfolder -rs settings/custom_settings.prop -n "PXD032759"  
--pulsarStage pulsarStep2
```

This command:

- Specifies the step using option `--pulsarStage pulsarStep2`.
- Takes all search archives from **Step 1.1** using the `-sa` option.
- Generates a .qsp file in the `intermediate/qsp` folder. The name of the file is `<Experiment_name>.qsp` (here, it would be `PXD032759.qsp`). `--noOutputSubfolder` option helps localize the file in an automated manner (without predicting the timestamp).
- Other options as described previously.

Parallelization workflow details

Final search archive file generation

Step 1.3

Create a final search archive from the intermediate search archive and .qsp file:

```
$ spectronaut lg -se Pulsar -r raw/<raw_path_i> ...  
-sa intermediate/search_archive_step1_file<i>.psar -a intermediate/search_archive_file<i>.psar  
--optimizedModels intermediate/qsp/PXD032759.qsp  
--pulsarStage pulsarStep3  
-o intermediate -n "PXD032759" -rs settings/custom_settings.prop
```

For the batch <i>, the command above:

- Takes one or more raw files selected for batch <i>: raw/<raw_path_i> as an input. Raw files can also be specified with the -d raw option, similarly to [Step 1.1](#).
- Takes an intermediate search archive from [Step 1.1](#) (intermediate/search_archive_step1_file<i>.psar) specified with the -sa option.
- Produces the final search archive (intermediate/search_archive_file<i>.psar) specified with -a option.
- Takes a .qsp file from [Step 1.2](#) with option --optimizedModels
- Specifies step with option --pulsarStage pulsarStep3
- Other options as previously described.

Parallelization workflow details

Library generation

Step 1.4

All final search archives are merged into one search library (.kit file):

```
$ spectronaut lg -se Pulsar  
-sa intermediate/search_archive_file1.psar -sa intermediate/search_archive2.psar ...  
-k intermediate/output_library.kit -o intermediate -s settings/custom_settings.prop
```

This command:

- merges search archives from **Step 1.3** into one library output_library.kit and places it in the intermediate folder (specified with the -a option). The library file will be used in the next step.
- Library generation settings are specified in a settings file, settings/custom_settings.prop.

Parallelization workflow details

DIA Analysis

Step 2.1

Search the raw data against the merged library

```
$ spectronaut diaanalysis -o intermediate -a intermediate/output_library.kit -d raw -n "PXD032759"  
-s settings/custom_settings.prop
```

The command above runs the DIA Analysis pipeline:

- Runs can be selected for a batch using the `-r` option or by selecting them in a raw folder and using the `-d` option.
- It produces an `.sne` file that can be combined in the next step. The `.sne` file is placed in a time-stamped subfolder of a results folder (`results/<timestamp>_<experiment_name>`).
- Other settings as specified for **Step 2**.

Parallelization workflow details

Merging .sne files

Step 2.2

Merge .sne files on an experiment level to produce a single .sne file and a report

```
$ spectronaut manageSNE --merge -o results -sne intermediate/<subfolder1>/<timestamp>_PXD032759.sne  
-sne intermediate/<subfolder2>/<timestamp>_PXD032759.sne ...  
-o results -s settings/custom_settings.prop -con other_files/PXD032759_condition_setup.tsv
```

The command above:

- Merges multiple .sne files on an experiment level and saves the final .sne file and a report in a timestamped subfolder of the results folder
- .sne files can be specified with the -sne option. intermediate/<subfolder1>/ structure reflects the folder structure from the previous step.
- You can specify multiple .sne files to be merged. Also, you can use the -d <folder> option to merge all .sne files in a folder. Please run `$ spectronaut manageSNE -h` for more details.
- For large experiments (typically greater than 500 samples), merging them may exceed available memory and disk space (could be greater than a terabyte for each). In such a scenario, use the combine command instead (see next slide).
- Run conditions are specified in a .tsv file,
- One can overwrite post-process settings using a settings file.

Parallelization workflow details

Combining .sne files for large experiments

For large experiments (typically greater than 500 samples), merging them may exceed available memory and disk space. In such a scenario, use the `combine` command instead, as below.

Step 2.2

Combine .sne files on an experiment level to produce a single report

```
$ spectronaut combine -o results -sne intermediate/<subfolder1>/<timestamp>_PXD032759.sne  
-sne intermediate/<subfolder2>/<timestamp>_PXD032759.sne ...  
-fasta fasta/20210709-SP_mouse_isoforms_UP000000589.fasta -fasta fasta/crap.2009.05.01.fasta  
-s settings/custom_settings.prop
```

The command above:

- Combines multiple .sne files on an experiment level and saves the final report in a results folder
- `intermediate/<subfolder1>/` structure reflects folder structure from the previous step
- FASTA files are used as specified,
- You can specify multiple .sne files to be combined. Also, you can use the `-d <folder>` option to combine all .sne files in a folder. Please run `$ spectronaut combine -h` for more details.

Additional notes

Spectronaut version applicability

- ❖ The pipeline described here is applicable for Spectronaut version ≥ 20.3 .
- ❖ For older versions of Spectronaut, a simplified parallelization pipeline is applicable. Please contact the Biognosys Product Support Team at support@biognosys.com for details.

Suggested hardware requirements

We suggest the following as a starting point and we recommend testing to optimize for your needs

Pipeline	RAM	CPU	Hard disk
„Mapping” pipelines: - Search archive generation (Steps 1.1, 1.3) - DIA Analysis	32 GB	32 to 64 cores	5x sum of a sum of input MS file sizes
„Reducing” pipelines: - Archive merging - SNE combine .qsp file generation	256 GB – 512 GB for 2000 samples	16 to 64 cores	3x the sum of all input artifacts
„Reducing” pipeline: SNE merge	Uses significantly more RAM. Please see the previous slides for guidelines.	16 to 32 cores	3x the sum of all input SNE artifacts

Summary of most important files to be handled

Step	Required input files	Most important output files
1.1	- Raw files, e.g., .d, .raw (for every node, only the files analyzed by the node are needed) - Fasta files - Additional files if needed	Search archive, .psar (single file for each node)
1.2	Search archives from Step 1.1 (files from all nodes)	A .qsp file
1.3	- Raw files, e.g., .d, .raw (for every node, only the files analyzed by the node are needed) - A .qsp file from Step 1.2	Search archive, .psar (single file for node)
1.4	Search archives from Step 1.3 (files from all nodes)	A library, .kit
2.1	- Raw files, e.g., .d, .raw (for every node, only the files analyzed by the node are needed) - Library from Step 1.4	Spectronaut experiment file, .sne (single file for each node)
2.2	Spectronaut experiment files from Step 2.1 (files from all nodes)	Spectronaut experiment file, .sne (single file for whole experiment)

Changing settings

- ❖ For Spectronaut Linux, the most important search settings can be changed using a .json file, which can be provided using the `-j` option of the `lg` or `diaanalysis` pipelines.
- ❖ A settings .json file with default settings values is available at https://files.biognosys.ch/FileSharing/FUKTgiAxv1WzIYrM/directDIA_settings.json.

Spectronaut activation and licensing

Types of licenses

Classical licensing model

- ❖ Every token is bound to a machine, needs to be deactivated to release the seat
- ❖ Containers considered as separate machines
- ❖ It works best with physical machines or when Spectronaut is not moving from machine to machine
- ❖ Not recommended for containerized environment

Floating license model

- ❖ The license is bound to a machine, but can be used on several ones; the token is taken only for Spectronaut execution
- ❖ Tokens are freed automatically after Spectronaut execution
- ❖ Needs an internet connection during Spectronaut execution
- ❖ The number of tokens limits the number of running Spectronaut instances at the same time
- ❖ Recommended for a containerized environment

Floating licensing model and containers

- ❖ Every container is considered a different machine; the classic license needs to be manually deactivated always before the container is torn down
- ❖ Use a floating license instead. When starting a container:

```
$ spectronaut activate <license_key>  
$ spectronaut ... <processing commands>
```

- ❖ This command will connect the licensing server to bound the machine with the license key; the token will be taken automatically.
- ❖ Do not deactivate without an explicit purpose; it may block tokens or slow down the licensing process.
- ❖ Starting with Spectronaut 19.8, activation can be done once in the container. For the next run, Spectronaut will check the presence of the `license.json` file at `/mnt/bgs-license` directory (has to be mounted by the user for every execution of a container).
- ❖ Alternatively, for floating licensing, the activation can be done once in the image and committed to the image (however, *not* best practice for secrets storage purposes)

TRANSFORMATIVE INSIGHTS FROM DISCOVERY TO CLINIC



www.biognosys.com